

Extended / deep thinking

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 9 of 12 · English notes

20 August opened the “advanced features” hour: everything is still **context enhancement** on a stateless Messages call. First feature: **extended thinking** (class: deep thinking). You do not have to few-shot a chain of thought in the user prompt. You set a request parameter so Claude allocates a **thinking budget**, produces a thinking block, then the visible answer.

What it is for

User message example: “Write a short guide on recursion.” With thinking enabled, the model first plans (what recursion is, how it works, structure of the guide) and only then writes the guide. Vijay called that plan an internal chain of thought. Output quality can rise on hard tasks; cost and latency rise with it.

How you enable it on create

Alongside model, max_tokens, temperature: a **thinking** object. Class notebook: if thinking is on, send type: enabled plus a **budget** in tokens. He taught a **minimum of 1024** thinking tokens. Raise the budget when the problem is harder. Those thinking tokens are generated content and were described as **billed like output** — “exponentially” more expensive than a plain short answer if you always leave thinking on.

MAX_TOKENS VERSUS THINKING BUDGET

Official docs require max_tokens to be large enough for thinking budget *plus* the visible answer. If class code and docs disagree, follow **extended thinking**. Do not set thinking budget above max_tokens.

Two blocks in the assistant message

Block	What you see	What it is for
thinking	Plain-language plan (when not redacted)	Inspect or replay in later turns
redacted_thinking	Encrypted / opaque payload	Safety: model withholds sensitive reasoning; you cannot decode it
text	The user-visible answer	What you show in the UI

Without thinking, content was a single text block. With thinking, you iterate blocks by type. A “magic string” in the lab forced redacted thinking so students could see the encrypted shape; day-to-day you do not put that string in production prompts. Redaction is the model’s call when reasoning would leak something it treats as sensitive (Vijay analogised “business secrets” of the model / DMZ-like internals).

You cannot decrypt redacted thinking

Mritunjay asked if clients can decode it. Vijay: no — only the server has the cipher. Purpose of sending it back: **multi-turn**. Next request should include the previous thinking (including redacted) so the model continues the same chain. If you need human-readable plans, do not trigger redaction, and/or write your own chain-of-thought examples in the prompt instead of relying on extended thinking.

Relationship to few-shot CoT

Classic CoT: you show “2+2=4 because...” then ask the real question. Extended thinking: the model writes its own plan. You can still add examples if you need a *particular* method; the built-in thinking may not match your domain rubric. If you are not confident, either raise budget, add examples, or skip thinking and specify steps yourself.

When not to use it

- Trivial lookups and short classifications (you pay for a 1024-token plan).
- Tight latency SLAs.
- When a schema / tool call already structures the work.

PRODUCT NAMES DRIFT

Class said “deep thinking”; docs say **extended thinking**. Token budgets, interleaved thinking, and which models support it change. Trust current docs over the 20 August notebook flags.

Official documentation

- **Extended thinking**

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

enabled + budget_tokens, redacted thinking, streaming.

- **Messages API**

<https://docs.claude.com/en/api/messages>

thinking parameter on create.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

Which SKUs support thinking.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.